

Diário de Engenharia: Provisionamento, Troubleshooting e Validação do Oracle Database 19c RAC em Oracle Linux 9

Autor: Arley Ribeiro *Técnico em Informática | Suporte, Banco de Dados & SQL* **Data:** Setembro de 2026 **Tecnologias:** Oracle Real Application Clusters (RAC) 19c (19.3), Oracle Automatic Storage Management (ASM), Oracle Grid Infrastructure, HashiCorp Vagrant, Oracle VirtualBox 7.0+, Oracle Linux 9 (UEK7 / Kernel 6.12, glibc 2.34).

Sumário

- 1. Introdução e Visão Geral da Arquitetura
- 2. Modelagem de Infraestrutura como Código (IaC) e Topologia de Rede
- 3. Armazenamento Compartilhado SCSI, Regras UDEV e Diskgroups ASM
- 4. Engenharia Reversa e Compatibilidade GLIBC 2.34 no Oracle Linux 9
- 5. Desafios do Clusterware: Sincronismo CTSS vs Chrony e CVU
- 6. Instalação do RDBMS e Provisionamento do Banco Multitenant via DBCA
- 7. Validação Operacional de Alta Disponibilidade e Consultas GV\$
- 8. Governança Git e Procedimento de Desligamento Gracioso
- 9. Conclusão e Lições Aprendidas

1. Introdução e Visão Geral da Arquitetura

O Oracle Real Application Clusters (RAC) representa a tecnologia de ponta da Oracle para banco de dados relacional distribuído com arquitetura *shared-everything*. A execução do Oracle 19c Enterprise Edition em nós virtualizados com o Oracle Linux 9 (OL9) apresenta desafios arquiteturais notáveis, pois a versão base 19.3.0 foi lançada para sistemas com glibc 2.28 (OL7/OL8), enquanto o OL9 traz a glibc 2.34 e o Kernel UEK7 (6.12).

Este documento relata as etapas de engenharia, os obstáculos enfrentados e as soluções de contorno concebidas para entregar um ambiente RAC de 2 nós (**racnode1** e **racnode2**) totalmente operacional.

Especificações Técnicas Centrais

Componente	Detalhe da Configuração
Sistema Operacional	Oracle Linux 9 (UEK7 / Kernel 6.12 x86_64)
Clusterware / Grid	Oracle Grid Infrastructure 19c (19.3.0.0.0 Enterprise Edition)
RDBMS	Oracle Database 19c (19.3.0.0.0 Enterprise Edition Multitenant)
CDB / PDB	Container Database orcl com Pluggable Database orclpdb
Armazenamento	Oracle Automatic Storage Management (ASM) com discos SCSI compartilhados
Recursos por Nó	6144 MB (6 GB) de RAM, 4096 MB de Swap e 2 vCPUs

2. Modelagem de Infraestrutura como Código (IaC) e Topologia de Rede

Para garantir a reprodutibilidade total do cluster, o provisionamento foi automatizado via HashiCorp Vagrant acoplado ao Oracle VirtualBox.

Extensões do Hypervisor e Plugins do Orquestrador

- VirtualBox Extension Pack:** Pré-requisito mandatório instalado no host na mesma versão do VirtualBox. Fornece o backend de emulação para barramentos NVMe, controladoras SCSI avançadas e recursos compartilhados essenciais para que o VirtualBox gerencie discos com o descritor **shareable** sem causar locks exclusivos no sistema de arquivos hospedeiro.
- Plugin Vagrant **vagrant-vbguest**:** Recomendado para sincronizar automaticamente as *VirtualBox Guest Additions* dentro do ambiente Oracle Linux 9 (UEK7 / Kernel 6.12), mantendo a compatibilidade dos módulos de integração do kernel com o hipervisor.

Topologia Multi-Homed por VM

Cada nó do cluster conta com 3 adaptadores de rede físicos/virtuais:

1. **Interface 1 (eth0 - NAT)**: Dedicada à gerência do Vagrant, redirecionamento de portas e conectividade externa à internet.
2. **Interface 2 (eth1 - Host-Only 192.168.56.x)**: Rede pública do cluster, responsável pelo tráfego de clientes, VIPs de failover e Single Client Access Name (SCAN).
3. **Interface 3 (eth2 - Internal Network rac_priv_net 192.168.10.x)**: Rede isolada do VirtualBox dedicada exclusivamente ao Cluster Interconnect, Cache Fusion e High Availability IP (HAIP).

Tabela de Endereçamento IP

Hostname	Interface	IP / Máscara	Finalidade
racnode1	eth1	192.168.56.11 / 24	IP Público / Gestão Nó 1
racnode1-vip	eth1:1	192.168.56.21 / 24	Virtual IP Nó 1
racnode1-priv	eth2	192.168.10.11 / 24	Interconnect Nó 1
racnode2	eth1	192.168.56.12 / 24	IP Público / Gestão Nó 2
racnode2-vip	eth1:1	192.168.56.22 / 24	Virtual IP Nó 2
racnode2-priv	eth2	192.168.10.12 / 24	Interconnect Nó 2
rac-scan	DNS (dnsmasq)	192.168.56.31, .32, .33	SCAN VIPs Round-Robin

3. Armazenamento Compartilhado SCSI, Regras UDEV e Diskgroups ASM

O Oracle ASM exige que todos os nós do cluster tenham acesso de leitura e escrita simultâneo aos mesmos blocos de disco físico.

Criação dos Discos Compartilhados (VirtualBox + Extension Pack)

No **Vagrantfile**, os discos foram definidos no diretório **.asm_storage/** com o atributo **shareable** acoplados a uma controladora SCSI independente, eliminando o bloqueio de arquivo no host Windows graças às primitivas de I/O providas pelo VirtualBox Extension Pack.

Diskgroups ASM Criados

1. **+CRS (15 GB - Redundância NORMAL)**:
 - Composto por 3 discos de 5 GB (**asm-crs01**, **asm-crs02**, **asm-crs03**).
 - Armazena o Oracle Cluster Registry (OCR) e os 3 Voting Disks quorum.
2. **+DATA (30 GB - Redundância EXTERNAL)**:
 - Composto pelo disco **asm-data01** (30 GB).
 - Armazena Datafiles do CDB e do PDB, Controlfiles, Redo Logs e Tablespace SYSTEM/SYSAUX.
3. **+RECO (20 GB - Redundância EXTERNAL)**:
 - Composto pelo disco **asm-reco01** (20 GB).
 - Armazena a Fast Recovery Area (FRA), cópias de multiplexação e Archive Logs.

Regras UDEV para Persistência de Permissões

Para garantir a posse **grid:asmadmin** e permissões **0660** persistentes entre reboots:

```
# /etc/udev/rules.d/99-oracle-asmdevices.rules
KERNEL=="sd[b-f]", OWNER="grid", GROUP="asmadmin", MODE="0660"
```

4. Engenharia Reversa e Compatibilidade GLIBC 2.34 no Oracle Linux 9

O Oracle Linux 9 adota a biblioteca C **glibc 2.34**, a qual incorporou funções clássicas de manipulação de POSIX threads diretamente no **libc.so.6**, extinguindo o arquivo estático **libpthread_nonshared.a**. Adicionalmente, as chamadas de sistema **stat**, **lstat** e **fstat** foram convertidas para versões versionadas **__xstat**.

1. Problema de Linkagem do Grid & RDBMS

Durante a execução do **gridSetup.sh** e **runInstaller**, a etapa de compilação falhava com erros de símbolos indefinidos:

- **/usr/bin/ld: cannot find -lpthread_nonshared: No such file or directory**
- **undefined reference to stat, undefined reference to lstat**

2. Solução Técnica Desenvolvida

1. Criação do Stub `libpthread_nonshared.a`:

```
sudo ar cr /usr/lib64/libpthread_nonshared.a
```

2. Desenvolvimento da Biblioteca Shim (`libstat_shim.a`):

Compilação de um wrapper em C que redireciona as chamadas legadas para as APIs modernas do kernel:

```
#include <sys/stat.h>
int stat(const char *path, struct stat *buf) { return __xstat(1, path, buf); }
int lstat(const char *path, struct stat *buf) { return __lxstat(1, path, buf); }
int fstat(int fd, struct stat *buf) { return __fxstat(1, fd, buf); }
```

```
gcc -c -fPIC stat_shim.c -o stat_shim.o
ar cr /usr/lib64/libstat_shim.a stat_shim.o
```

3. Injeção no `sysliblist`:

Adicionamos a flag `-lstat_shim` aos arquivos de definição de linkagem:

- `$GRID_HOME/lib/sysliblist`
- `$ORACLE_HOME/lib/sysliblist`

Com isso, a compilação do Grid Infrastructure e do Oracle Database concluiu com **100% de sucesso**.

5. Desafios do Clusterware: Sincronismo CTSS vs Chrony e CVU

1. Mascaramento do Chronyd para o CTSS

No Oracle Clusterware, o *Cluster Time Synchronization Service* (CTSS) monitora o desvio de relógio entre os nós. Se o serviço `chronyd` estiver em execução desordenada, o CTSS entra em modo **OBSERVER** ou gera alertas de split-brain no CSSD.

- **Ação:** O serviço `chronyd` foi desativado e mascarado em ambos os nós:

```
sudo systemctl stop chronyd
sudo systemctl disable chronyd
sudo systemctl mask chronyd
```

- **Resultado:** O CTSS assumiu o papel ativo (**ACTIVE:0, STABLE**) com sincronismo perfeito entre nós.

2. Contorno do Cluster Verification Utility (CVU)

Como a versão 19.3 não reconhecia oficialmente a distribuição **OL9**, o instalador abortava na verificação de distribuição.

- **Solução:** Exportação da variável de compatibilidade e uso das flags de override:

```
export CV_ASSUME_DISTID=OL8
./gridSetup.sh -silent -responseFile /vagrant/templates/grid_install.rsp
./runInstaller -silent -ignorePrereq ...
```

6. Instalação do RDBMS e Provisionamento do Banco Multitenant via DBCA

Após a estabilização da infraestrutura do Grid e montagem dos diskgroups ASM, executou-se a instalação dos binários do RDBMS e a criação do banco de dados multitenant.

1. Instalação dos Binários do Database

Extração no `$ORACLE_HOME (/u01/app/oracle/product/19.0.0/dbhome_1)` no `racnode1` e execução do instalador silencioso com propagação para `racnode2`.

2. Criação Silenciosa do Banco RAC Multitenant

A criação foi realizada via `dbca` gerenciando a alocação de memória para 2048 MB, de modo a preservar os limites de RAM das máquinas virtuais:

```

dbca -silent -createDatabase \
  -templateName General_Purpose.dbc \
  -gdbName orcl \
  -sid orcl \
  -databaseConfigType RAC \
  -odelist racnode1,racnode2 \
  -createAsContainerDatabase true \
  -numberOfPDBs 1 \
  -pdbName orclpdb \
  -sysPassword "Oracle_1234_#" \
  -systemPassword "Oracle_1234_#" \
  -pdbAdminPassword "Oracle_1234_#" \
  -storageType ASM \
  -datafileDestination +DATA \
  -recoveryAreaDestination +RECO \
  -characterSet AL32UTF8 \
  -nationalCharacterSet AL16UTF16 \
  -totalMemory 2048 \
  -redoLogFileSize 100 \
  -ignorePrereqFailure -ignorePreReqs

```

7. Validação Operacional de Alta Disponibilidade e Consultas GV\$

Ao término da criação, validamos o estado do cluster e do banco multitenant através dos utilitários **srvctl**, **crsctl** e consultas SQL no dicionário de dados.

1. Status do Banco via **srvctl**

```

Instance orcl1 is running on node racnode1. Instance status: Open.
Instance orcl2 is running on node racnode2. Instance status: Open.

```

2. Consulta de Instâncias (**GV\$INSTANCE**)

```
SELECT inst_id, instance_name, host_name, status, startup_time FROM gv$instance ORDER BY inst_id;
```

INST_ID	INSTANCE_NAME	HOST_NAME	STATUS
1	orcl1	racnode1.localdomain	OPEN
2	orcl2	racnode2.localdomain	OPEN

3. Consulta do Pluggable Database (**GV\$PDBS**)

```
SELECT inst_id, con_id, name, open_mode FROM gv$pdb WHERE name = 'ORCLPDB' ORDER BY inst_id;
```

INST_ID	CON_ID	NAME	OPEN_MODE
1	3	ORCLPDB	READ WRITE
2	3	ORCLPDB	READ WRITE

4. Status Consolidado do Clusterware (**crsctl stat res -t**)

Todos os recursos (**ora.LISTENER.lsnr**, **ora.CRS.dg**, **ora.DATA.dg**, **ora.RECO.dg**, **ora.asm**, **ora.orcl.db**, **ora.racnode1.vip**, **ora.racnode2.vip**, **ora.scan1.vip**) com status **ONLINE** e **STABLE**.

8. Ciclo de Vida do Cluster, Operação e Governança Git

1. Governança no **.gitignore**

Para manter o repositório leve, seguro e em conformidade:

- Discos virtuais (***.vdi**, ***.vmdk**), pastas **.vagrant/** e **.asm_storage/** foram estritamente excluídos.
- Instaladores e arquivos binários compactados da Oracle (**software/**, ***.zip**) foram ignorados.
- Logs e dumps transitórios (***.log**, ***.trc**, ***.trm**) foram removidos do rastreamento.
- Evidências gráficas foram centralizadas e permitidas apenas em **docs/img/**.

2. Procedimento de Inicialização Segura (Cold Start)

Ao iniciar o ambiente com as VMs desligadas:

1. **vagrant up racnode1 --no-provision** (Inicializa o nó master, discos compartilhados e serviços CRS/ASM).
2. **vagrant up racnode2 --no-provision** (Adiciona o nó secundário ao cluster e ingressa na comunicação Interconnect).
3. Validação inline: **vagrant ssh racnode1 -- -t "sudo su - grid -c 'crsctl stat res -t'"**

3. Procedimento de Reinicialização Controlada (Reboot dos Nós)

Para testar a resiliência do Clusterware e a persistência das regras UDEV:

1. Reiniciar o nó 2: **vagrant reload racnode2 --no-provision**
2. Reiniciar o nó 1: **vagrant reload racnode1 --no-provision**
3. Após a convergência da stack, todos os recursos (**ora.orcl.db**, VIPs, SCAN e ASM) restabelecem automaticamente o status **ONLINE**.

4. Procedimento de Desligamento Gracioso (Graceful Shutdown)

Para prevenir corrupção no ASM, nos blocos do OCR e nos voting disks:

1. Parada do Banco de Dados Multitenant:

```
vagrant ssh racnode1 -- -t "sudo su - oracle -c 'srvctl stop database -d orcl -o immediate'"
```

2. Parada do Clusterware (CRS) em cascata:

```
vagrant ssh racnode2 -- -t "sudo su - root -c '/u01/app/19.0.0/grid/bin/crsctl stop crs -f'"  
vagrant ssh racnode1 -- -t "sudo su - root -c '/u01/app/19.0.0/grid/bin/crsctl stop crs -f'"
```

3. Desligamento das Máquinas Virtuais:

```
vagrant halt racnode2  
vagrant halt racnode1
```

4. Confirmação do estado no hypervisor:

```
vagrant status
```

9. Conclusão e Lições Aprendidas

O provisionamento do Oracle Database 19c RAC em nós virtualizados com Oracle Linux 9 comprovou que a incompatibilidade aparente de versões pode ser superada com engenharia de compilação (shims em C), governança precisa de memória no hypervisor e orquestração determinística via UDEV e Vagrant.

O ambiente resultante encontra-se estável, versionado no Git e pronto para ser utilizado como laboratório avançado de estudos ou base de testes de alta disponibilidade.

Arley Ribeiro [LinkedIn](#) | [GitHub](#)